

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO
PHÁT TRIỂN ỨNG DỤNG WEB
CT449 – 01

Giảng viên hướng dẫn:

ThS Nguyễn Minh Trung

Sinh viên thực hiện:

Tên: Lâm Thành Đô

MSSV: B2306613

Cần Thơ, năm 2026

MỤC LỤC

BACKEND 1	1
I. Cài đặt môi trường node và git	1
II. Tạo thư mục Node	1
III. Quản lí mã nguồn với git và GitHub	2
IV. Cấu hình Visual Studio Code, ESLint và Prettier	4
V. Cài đặt Express	5
VI. Định nghĩa Controller và các route	9
VII. Cài đặt xử lý lỗi.	12
LINK REPO BACKEND1:.....	15
BACKEND 2	16
I. Cài đặt và Khởi động MongoDB.....	16
II. Cài đặt thư viện mongodb, định truy xuất cơ sở dữ liệu	16
III. Cài đặt các handler	18
LINK REPO BACKEND2:	29

DANH MỤC HÌNH

Hình 1: Kiểm tra phiên bản Node.js và Git.....	1
Hình 2: Khởi tạo dự án bằng lệnh npm init.....	1
Hình 3: Cài đặt các thư viện Express và CORS	2
Hình 4: Khởi tạo Git cục bộ và kiểm tra trạng thái tệp	2
Hình 5: Thực hiện lưu mã nguồn (commit) lần đầu.....	2
Hình 6: Đẩy mã nguồn lên kho lưu trữ GitHub	3
Hình 7: Giao diện mã nguồn dự án trên GitHub	3
Hình 8: Cài đặt thư viện ESLint và Prettier	4
Hình 9: Cấu hình các quy tắc trong tệp .eslintrc.js.	4
Hình 10: Cài đặt Extension ESLint trên VS Code	4
Hình 11: Cài đặt Extension Prettier trên VS Code.....	5
Hình 12: Lưu cấu hình ESLint vào Git	5
Hình 13: Nội dung tệp app.js khởi tạo Express.....	5
Hình 14: Nội dung tệp server.js thiết lập máy chủ.....	6
Hình 15: Cấu hình cổng (port) trong app/config/index.js	6
Hình 16: Thông báo máy chủ đang chạy trên cổng 3000.....	6
Hình 17: Kiểm tra phản hồi của server trên trình duyệt.....	7
Hình 18: Cấu hình mục scripts trong tệp tin package.json.....	7
Hình 19: Chạy server bằng lệnh npm start.	8
Hình 20: Lưu các thay đổi về Express vào Git.....	8
Hình 22: Thiết lập các tuyến đường trong contact.route.js.	10
Hình 23: Đăng ký các route vào ứng dụng chính.....	10
Hình 24: Kiểm tra route lấy danh sách bằng Postman.	11

Hình 25: Kiểm tra route tạo mới bằng Postman.....	11
Hình 26: Lưu định nghĩa route vào Git.	12
Hình 27: Tạo lớp ApiError để quản lý lỗi.	12
Hình 28: Cấu hình middleware xử lý lỗi trong app.js.	13
Hình 29: Kiểm tra phản hồi API thành công.....	13
Hình 30: Kiểm tra xử lý lỗi khi sai đường dẫn (404).	14
Hình 31: Đẩy mã nguồn xử lý lỗi lên GitHub.....	14
Hình 32: Cấu trúc thư mục dự án sau phần Backend 1.....	15
Hình 33: Giao diện quản lý MongoDB Compass.	16
Hình 34: Cài đặt thư viện mongodb vào dự án.	16
Hình 35: Cấu hình kết nối MongoDB trong tệp config.....	17
Hình 36: Lớp tiện ích kết nối cơ sở dữ liệu mongodb.util.js.	17
Hình 37: Cập nhật logic khởi động server có kết nối DB.	18
Hình 38: Khởi tạo lớp dịch vụ ContactService.	18
Hình 39: Cài đặt hàm xử lý tạo liên hệ trong controller.	19
Hình 40: Xây dựng logic tạo liên hệ trong service.	19
Hình 41: Thiết lập kiểu dữ liệu gửi đi trên Postman.	20
Hình 42: Kiểm tra tính năng tạo liên hệ bằng Postman.	20
Hình 43: Cài đặt hàm lấy tất cả liên hệ trong controller.	20
Hình 44: Xây dựng logic tìm kiếm trong service.....	21
Hình 45: Kiểm tra tìm kiếm theo tên bằng Postman.	21
Hình 46: Cài đặt hàm lấy một liên hệ trong controller.....	22
Hình 47: Xây dựng logic tìm kiếm theo ID trong service.....	22
Hình 48: Kiểm tra lấy chi tiết liên hệ bằng Postman.	23
Hình 49: Cài đặt hàm cập nhật trong controller.	23

Hình 50: Xây dựng logic cập nhật dữ liệu trong service.....	24
Hình 51: Thông báo cập nhật thành công trên Postman.	24
Hình 52: Kiểm tra dữ liệu sau khi cập nhật.....	25
Hình 53: Cài đặt hàm xóa liên hệ trong controller.....	25
Hình 54: Xây dựng logic xóa dữ liệu trong service.	26
Hình 55: Kiểm tra tính năng xóa bằng Postman.	26
Hình 56: Cài đặt hàm lọc liên hệ yêu thích trong controller.....	26
Hình 58: Kiểm tra danh sách yêu thích bằng Postman.	27
Hình 59: Cài đặt hàm xóa tất cả trong controller.	27
Hình 60: Xây dựng logic xóa tất cả trong service.....	28
Hình 61: Kiểm tra tính năng xóa tất cả bằng Postman.....	28
Hình 62: Cập nhật thay đổi phần Backend 2 lên GitHub.....	28
Hình 63: Cấu trúc thư mục hoàn thiện của dự án.....	29

BACKEND 1

I. Cài đặt môi trường node và git

- Thực hiện tải nodejs và git theo hướng, sau đó kiểm tra bằng lệnh như ảnh:

```
PS D:\CT449> node -v
v24.14.0
PS D:\CT449> git --version
git version 2.53.0.windows.1
PS D:\CT449> |
```

Hình 1: Kiểm tra phiên bản Node.js và Git

II. Tạo thư mục Node

- Tạo thư mục dự án có tên Backend1, thực hiện *npm init*:

```
PS D:\CT449\Backend1> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help init' for definitive documentation on these fields
and exactly what they do.

Use 'npm install <pkg>' afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (backend1)
version: (1.0.0)
description:
entry point: (index.js) server.js
test command:
git repository:
keywords:
author: Thanh Do
license: (ISC)
type: (commonjs)
About to write to D:\CT449\Backend1\package.json:
{
  "name": "backend1",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Thanh Do",
  "license": "ISC",
  "type": "commonjs"
}
Is this OK? (yes)
```

Hình 2: Khởi tạo dự án bằng lệnh *npm init*

- Cài package cần thiết:

```
PS D:\CT449\Backend1> npm install express cors
added 67 packages, and audited 68 packages in 3s
23 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
```

Hình 3: Cài đặt các thư viện Express và CORS

III. Quản lý mã nguồn với git và GitHub

- Tạo tập tin .gitignore
- Khởi tạo Git cục bộ

```
PS D:\CT449\Backend1> npx gitignore node
Created .gitignore file for flag type node.
PS D:\CT449\Backend1> git init
Initialized empty Git repository in D:/CT449/Backend1/.git/
PS D:\CT449\Backend1> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        package-lock.json
        package.json

nothing added to commit but untracked files present (use "git add" to track)
```

Hình 4: Khởi tạo Git cục bộ và kiểm tra trạng thái tệp

- Thực thi git commit -m "Cai dat du an" để lưu các thay đổi.

```
PS D:\CT449\Backend1> git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
PS D:\CT449\Backend1> git commit -m "Cai dat du an"
[master (root-commit) 9463104] Cai dat du an
3 files changed, 1014 insertions(+)
create mode 100644 .gitignore
create mode 100644 package-lock.json
create mode 100644 package.json
PS D:\CT449\Backend1> git status
On branch master
nothing to commit, working tree clean
```

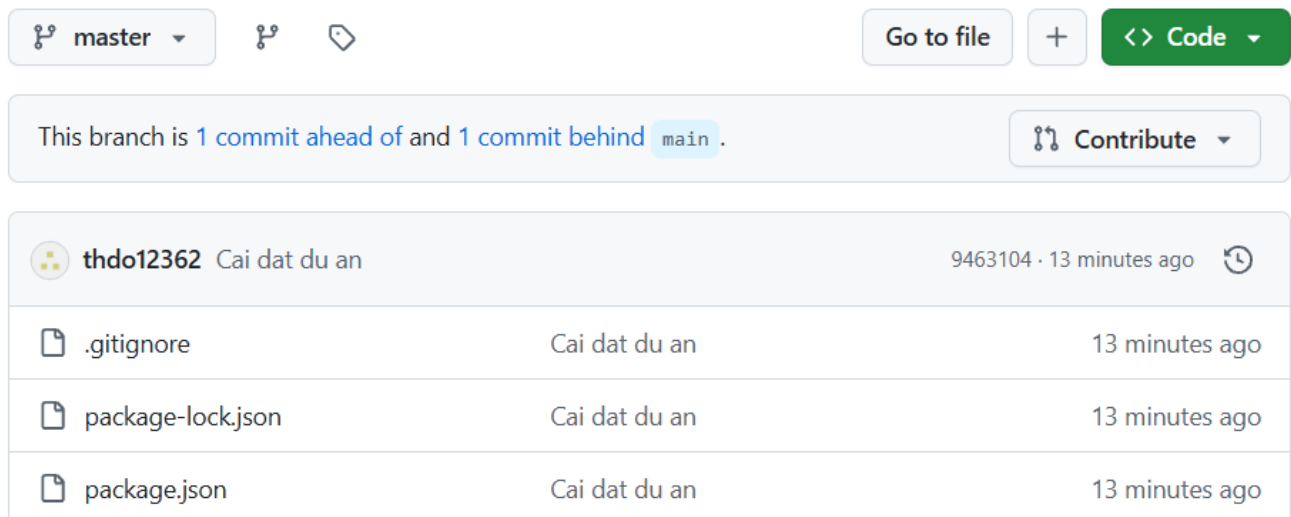
Hình 5: Thực hiện lưu mã nguồn (commit) lần đầu

- Đưa dự án lên GitHub

```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git remote add origin https://github.com/thdo12362/Backend1

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git push -u origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 8.79 KiB | 8.79 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/thdo12362/Backend1/pull/new/master
remote:
To https://github.com/thdo12362/Backend1
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Hình 6: Đẩy mã nguồn lên kho lưu trữ GitHub



master

Go to file + <> Code

This branch is 1 commit ahead of and 1 commit behind main.

Contribute

thdo12362 Cai dat du an 9463104 · 13 minutes ago

.gitignore	Cai dat du an	13 minutes ago
package-lock.json	Cai dat du an	13 minutes ago
package.json	Cai dat du an	13 minutes ago

Hình 7: Giao diện mã nguồn dự án trên GitHub

IV. Cấu hình Visual Studio Code, ESLint và Prettier

- Cài đặt các gói thư viện cần thiết

```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ npm i -D eslint prettier eslint-config-prettier

added 68 packages, and audited 136 packages in 4s

41 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

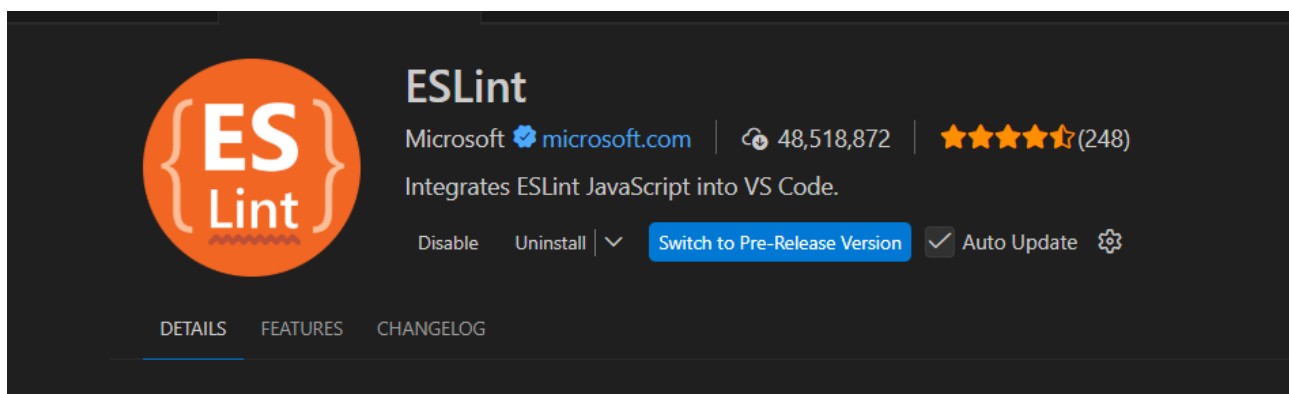
Hình 8: Cài đặt thư viện ESLint và Prettier

- Tạo tập tin cấu hình **.eslintrc.js** tại thư mục gốc

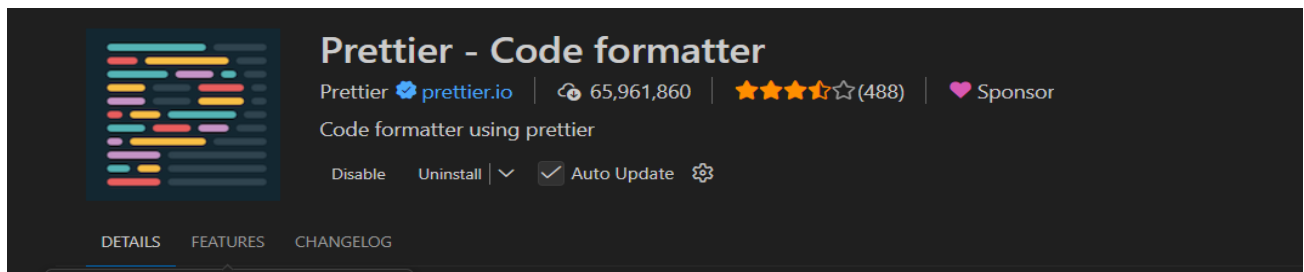
```
D: > CT449 > Backend1 > .eslintrc.js > ...
1  module.exports = {
2      env: {
3          node: true,
4          commonjs: true,
5          es2021: true,
6      },
7      extends: ['eslint:recommended', 'prettier'],
8  };
```

Hình 9: Cấu hình các quy tắc trong tập tin **.eslintrc.js**.

- Cài đặt Extension trên VS Code



Hình 10: Cài đặt Extension ESLint trên VS Code



Hình 11: Cài đặt Extension Prettier trên VS Code

- Lưu lại thay đổi vào Git

```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   package-lock.json
        modified:   package.json

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .eslinttrc.js

no changes added to commit (use "git add" and/or "git commit -a")

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add .
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git commit -m "Cau hinh ESLint cho du an"
[master 1a71a2d] Cau hinh ESLint cho du an
3 files changed, 874 insertions(+)
create mode 100644 .eslinttrc.js
```

Hình 12: Lưu cấu hình ESLint vào Git

V. Cài đặt Express

- Tạo tập tin *app.js* và *server.js* tại thư mục gốc

- *app.js*

```
D:\CT449 > Backend1 > JS app.js > ...
1  const express = require("express");
2  const cors = require("cors");
3
4  const app = express();
5
6  app.use(cors());
7  app.use(express.json());
8
9  app.get("/", (req, res) => {
10 |     res.json({ message: "Welcome to contact book application." });
11 | });
12
13 module.exports = app;
```

Hình 13: Nội dung tập tin *app.js* khởi tạo Express

- *server.js*

```
D: > CT449 > Backend1 > JS server.js > ...
1  const app = require("../app");
2  const config = require("../app/config");
3
4  const PORT = config.app.port;
5  app.listen(PORT, () => {
6    |   console.log(`Server is running on port ${PORT}.`);
7  });|
```

Hình 14: Nội dung tệp *server.js* thiết lập máy chủ

- Tạo cấu trúc thư mục *app/config* và file *index.js*

```
D: > CT449 > Backend1 > app > config > JS index.js > ...
1  const config = {
2    |   app: {
3    |     |   port: process.env.PORT || 3000,
4    |     |   }
5  };
6  module.exports = config;
```

Hình 15: Cấu hình cổng (port) trong *app/config/index.js*

- Thực hiện lệnh chạy server: *node server.js*. Truy cập <http://localhost:3000/> để kiểm tra kết quả

```
Thanh Do@DESKTOP-QOHDLAI MINGW64 /d/CT449/Backend1 (master)
$ node server.js
Server is running on port 3000.
```

← → ↻ ⓘ http://localhost:3000

Pretty-print ☐

```
{"message":"Welcome to contact book application."}
```

Hình 16: Thông báo máy chủ đang chạy trên cổng 3000

- Cài đặt thêm gói hỗ trợ nodemon để giám sát các thay đổi trên mã nguồn và tự động khởi động server: *npm install nodemon --save-dev*

```

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ npm install nodemon --save-dev

added 21 packages, and audited 157 packages in 1s

45 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

Hình 17: Kiểm tra phản hồi của server trên trình duyệt.

Mở file **package.json** và thay đổi cấu hình mục “scripts” như sau:

```

"scripts": {
  "start": "nodemon server.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},

```

```

D: > CT449 > Backend1 > {} package.json > {} dependencies
1  {
2    "name": "backend1",
3    "version": "1.0.0",
4    "description": "",
5    "license": "ISC",
6    "author": "Thanh Do",
7    "type": "commonjs",
8    "main": "server.js",
9    "scripts": {
10     "start": "nodemon server.js",
11     "test": "echo \"Error: no test specified\" && exit 1"
12   },
13   "dependencies": {},
14   "devDependencies": {
15     "cors": "^2.8.6",
16     "express": "^5.2.1"
17   },
18   "devDependencies": {
19     "eslint": "^10.0.2",
20     "eslint-config-prettier": "^10.1.8",
21     "nodemon": "^3.1.14",
22     "prettier": "^3.8.1"
23   }
24 }

```

Hình 18: Cấu hình mục scripts trong tập tin package.json.

- Với cấu hình trên ta có thể dùng lệnh **npm start** để chạy server thay vì **node server.js**

```

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ npm start

> backend1@1.0.0 start
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server is running on port 3000.

```

Hình 19: Chạy server bằng lệnh `npm start`.

– Lưu lại vào Git

```

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   package-lock.json
        modified:   package.json

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.js
        app/
        server.js

no changes added to commit (use "git add" and/or "git commit -a")

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add -u
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add app/ app.js server.js

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git commit -m "Cai dat express chay thong bao xin chao"
[master e063e49] Cai dat express chay thong bao xin chao
5 files changed, 316 insertions(+)
 create mode 100644 app.js
 create mode 100644 app/config/index.js
 create mode 100644 server.js

```

Hình 20: Lưu các thay đổi về Express vào Git.

VI. Định nghĩa Controller và các route

- Tạo thư mục *controllers* bên trong thư mục app. Sau đó tạo file *contact.controller.js* trong thư mục *controllers* với nội dung sau:

```
D: > CT449 > Backend1 > app > controllers > JS contact.controller.js > ...
1  exports.create = (req, res) => {
2    res.send({ message: "create handler" });
3  };
4
5  exports.findAll = (req, res) => {
6    res.send({ message: "findAll handler" });
7  };
8
9  exports.findOne = (req, res) => {
10   res.send({ message: "findOne handler" });
11 };
12
13 exports.update = (req, res) => {
14   res.send({ message: "update handler" });
15 };
16
17 exports.delete = (req, res) => {
18   res.send({ message: "delete handler" });
19 };
20
21 exports.deleteAll = (req, res) => {
22   res.send({ message: "deleteAll handler" });
23 };
24
25 exports.findAllFavorite = (req, res) => {
26   res.send({ message: "findAllFavorite handler" });
27 };|
```

Hình 21: Định nghĩa các hàm xử lý trong contact.controller.js.

- Tạo thư mục routes bên trong thư mục app. Sau đó tạo file *contact.route.js* trong thư mục routes với nội dung sau:

```
D: > CT449 > Backend1 > app > routes > JS contact.route.js > ...
1  const express = require("express");
2  const contacts = require("../controllers/contact.controller");
3  const router = express.Router();
4
5  router.route("/")
6    .get(contacts.findAll)
7    .post(contacts.create)
8    .delete(contacts.deleteAll);
9
10 router.route("/favorite")
11    .get(contacts.findAllFavorite);
12
13 router.route("/:id")
14    .get(contacts.findOne)
15    .put(contacts.update)
16    .delete(contacts.delete);
17
18 module.exports = router;
```

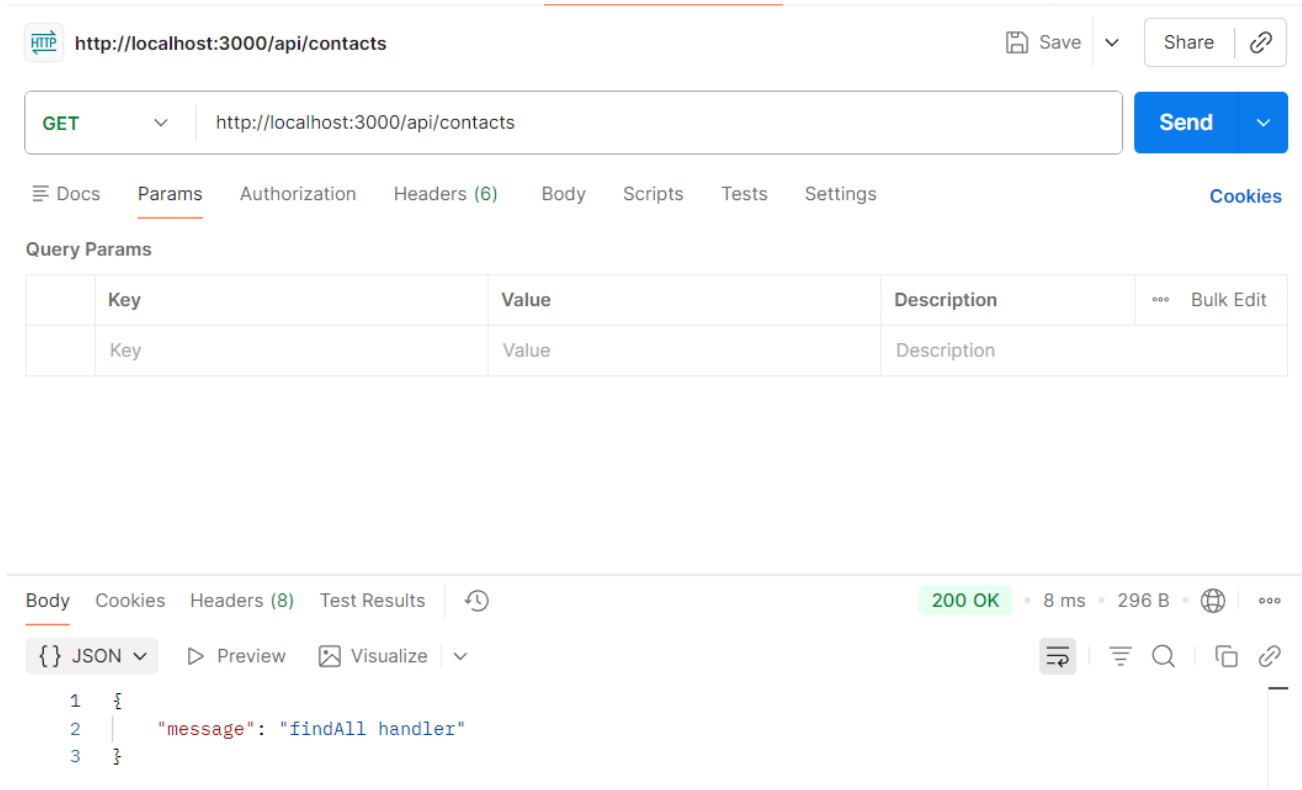
Hình 22: Thiết lập các tuyến đường trong *contact.route.js*.

- Đăng ký các route vừa tạo với express app. Hiệu chỉnh tập tin *app.js* như sau:

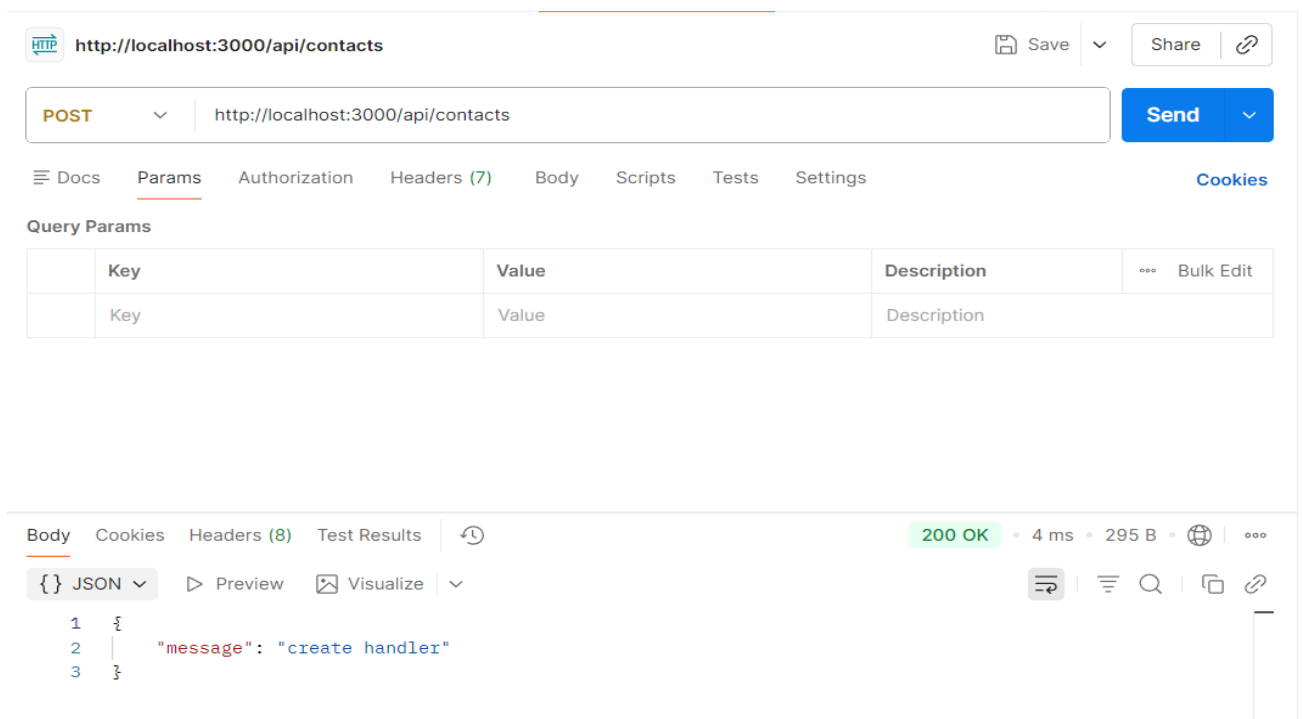
```
D: > CT449 > Backend1 > JS app.js > ...
1  const express = require("express");
2  const cors = require("cors");
3  const contactsRouter = require("../app/routes/contact.route"); // Thêm dòng này
4
5  const app = express();
6
7  app.use(cors());
8  app.use(express.json());
9
10 app.get("/", (req, res) => {
11   res.json({ message: "Welcome to contact book application." });
12 });
13
14 app.use("/api/contacts", contactsRouter); // Thêm dòng này
15
16 module.exports = app;
```

Hình 23: Đăng ký các route vào ứng dụng chính.

- Sử dụng ứng dụng Postman để kiểm tra các route vừa tạo và tiến hành lưu Git



Hình 24: Kiểm tra route lấy danh sách bằng Postman.



Hình 25: Kiểm tra route tạo mới bằng Postman.


```

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add -u

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add app/controllers app/routes

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git commit -m "Định nghĩa các route cho tài nguyên Contact"
[master delbea9] Định nghĩa các route cho tài nguyên Contact
3 files changed, 55 insertions(+), 7 deletions(-)
create mode 100644 app/controllers/contact.controller.js
create mode 100644 app/routes/contact.route.js

```

Hình 26: Lưu định nghĩa route vào Git.

VII. Cài đặt xử lý lỗi.

- Tạo tập tin *app/api-error.js*

```

D: > CT449 > Backend1 > app > JS api-error.js > ...
1  class ApiError extends Error {
2      constructor(statusCode, message) {
3          super();
4          this.statusCode = statusCode;
5          this.message = message;
6      }
7  }
8
9  module.exports = ApiError;
...

```

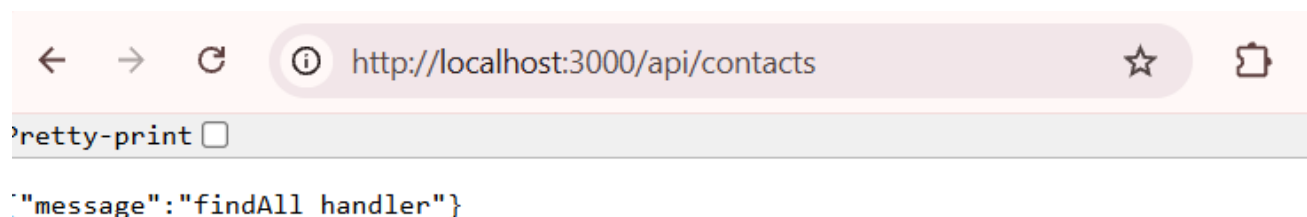
Hình 27: Tạo lớp *ApiError* để quản lý lỗi.

Mở file **app.js** và thêm vào các middleware xử lý lỗi như sau:

```
D: > CT449 > Backend1 > JS app.js > app.use() callback
1  const express = require("express");
2  const cors = require("cors");
3  const contactsRouter = require("../app/routes/contact.route");
4  const ApiError = require("../app/api-error"); // Thêm dòng này
5
6  const app = express();
7
8  app.use(cors());
9  app.use(express.json());
10
11 app.get("/", (req, res) => {
12   res.json({ message: "Welcome to contact book application." });
13 });
14
15 app.use("/api/contacts", contactsRouter);
16
17 // handle 404 response - Chạy khi không có route nào khớp yêu cầu
18 app.use((req, res, next) => {
19   return next(new ApiError(404, "Resource not found"));
20 });
21
22 // define error-handling middleware last, after other app.use() and routes calls
23 app.use((err, req, res, next) => {
24   // Middleware xử lý lỗi tập trung [
25   return res.status(err.statusCode || 500).json({
26     message: err.message || "Internal Server Error",
27   });
28   });
29
30 module.exports = app;
```

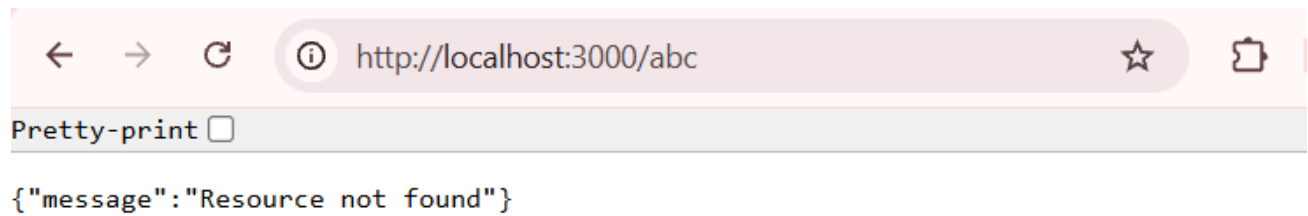
Hình 28: Cấu hình middleware xử lý lỗi trong app.js.

- Kiểm tra kết quả
 - Truy cập <http://localhost:3000/api/contacts>



Hình 29: Kiểm tra phản hồi API thành công.

- Truy cập một link bất kỳ như <http://localhost:3000/abc>



Hình 30: Kiểm tra xử lý lỗi khi sai đường dẫn (404).

- Lưu và đẩy code lên GitHub

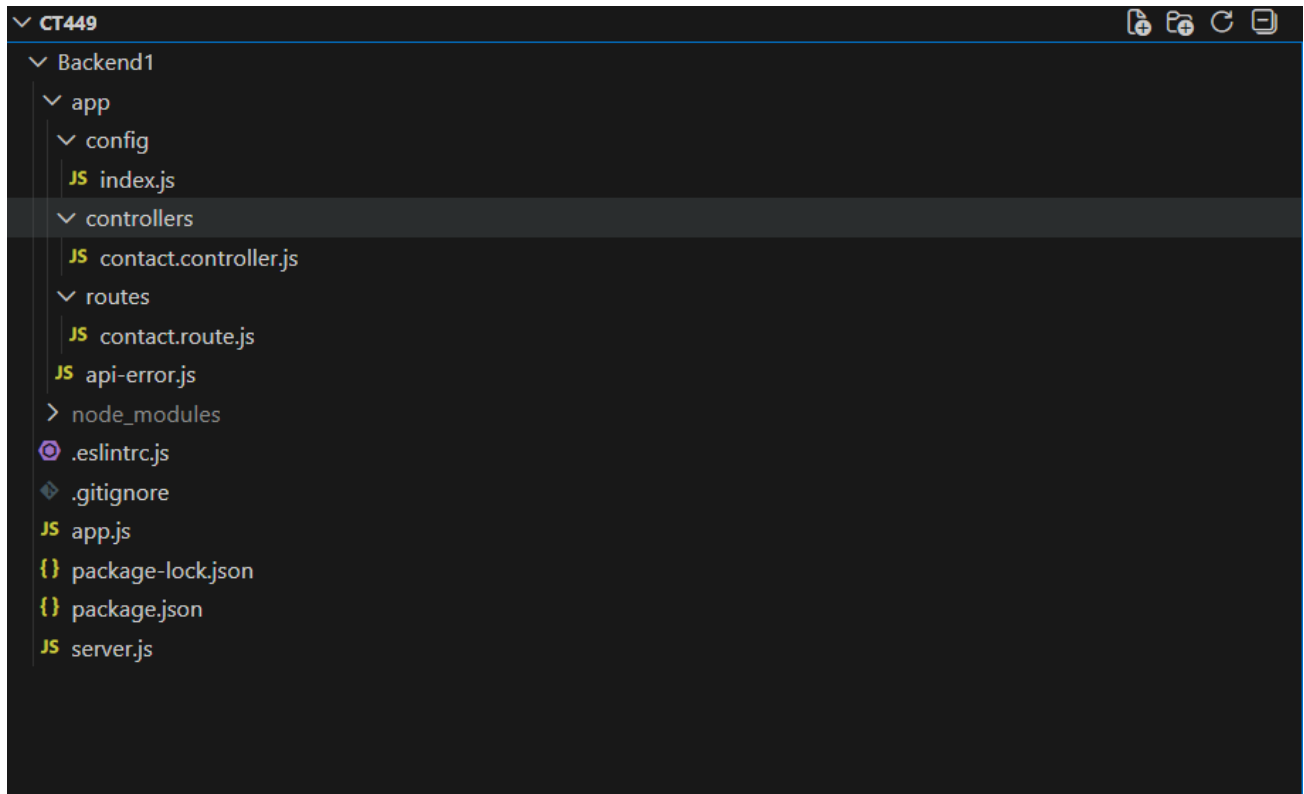
```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git add .

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git commit -m "Cai dat xu ly loi"
[master efae9e2] Cai dat xu ly loi
2 files changed, 25 insertions(+), 2 deletions(-)
create mode 100644 app/api-error.js

Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ git push origin master
Enumerating objects: 30, done.
Counting objects: 100% (30/30), done.
Delta compression using up to 16 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (27/27), 20.78 KiB | 6.93 MiB/s, done.
Total 27 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 1 local object.
To https://github.com/thdo12362/Backend1
9463104..efae9e2 master -> master
```

Hình 31: Đẩy mã nguồn xử lý lỗi lên GitHub.

Cấu trúc thư mục của dự án:



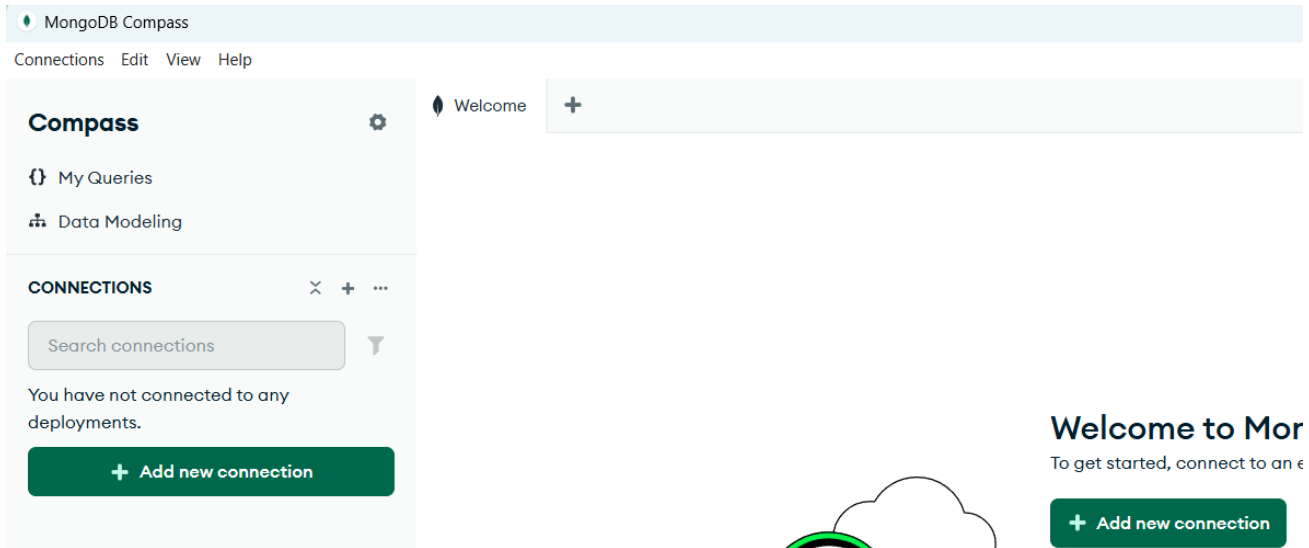
Hình 32: Cấu trúc thư mục dự án sau phần Backend 1.

LINK REPO BACKEND1:

<https://github.com/thdo12362/Backend1/tree/master>

BACKEND 2

I. Cài đặt và Khởi động MongoDB



Hình 33: Giao diện quản lý MongoDB Compass.

II. Cài đặt thư viện mongodb, định truy xuất cơ sở dữ liệu

- Cài đặt thư viện mongodb vào dự án: `npm install mongodb`

```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)
$ npm install mongodb

added 11 packages, and audited 168 packages in 3s

45 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Hình 34: Cài đặt thư viện mongodb vào dự án.

- Trong thư mục **app/config**, hiệu chỉnh (thêm đoạn code) file **index.js**

```
JS index.js M X
D: > CT449 > Backend1 > app > config > JS index.js > ...
1  const config = {
2    app: {
3      port: process.env.PORT || 3000,
4    },
5    db: {
6      uri: process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/contactbook"
7    }
8  };
9
10 module.exports = config;
```

Hình 35: Cấu hình kết nối MongoDB trong tệp config.

- Định nghĩa lớp trợ giúp kết nối đến MongoDB: **app/utils/mongodb.util.js**

```
D: > CT449 > Backend1 > app > utils > JS mongodb.util.js > ...
1  const { MongoClient } = require("mongodb");
2
3  class MongoDB {
4    static client;
5    static connect = async (uri) => {
6      if (this.client) return this.client;
7      this.client = await MongoClient.connect(uri);
8      return this.client;
9    };
10 }
11
12 module.exports = MongoDB;
```

Hình 36: Lớp tiện ích kết nối cơ sở dữ liệu mongodb.util.js.

- Cập nhật file chạy Server (server.js)

```
D: > CT449 > Backend1 > JS server.js > ...
1  const app = require("./app");
2  const config = require("./app/config");
3  const MongoDB = require("./app/utils/mongodb.util");
4
5  async function startServer() {
6      try {
7          await MongoDB.connect(config.db.uri);
8          console.log("Connected to the database!");
9
10         const PORT = config.app.port;
11         app.listen(PORT, () => {
12             console.log(`Server is running on port ${PORT}`);
13         });
14     } catch (error) {
15         console.log("Cannot connect to the database!", error);
16         process.exit();
17     }
18 }
19
20 startServer();
```

Hình 37: Cập nhật logic khởi động server có kết nối DB.

- Định nghĩa lớp dịch vụ ContactService (trong tập tin **app/services/contact.service.js**) chứa các API của thư viện mongodb để thực hiện các thao tác với CSDL MongoDB:

```
D: > CT449 > Backend1 > app > services > JS contact.service.js > ...
1  const { ObjectId } = require("mongodb");
2
3  class ContactService {
4      constructor(client) {
5          this.Contact = client.db().collection("contacts");
6      }
7      // Các phương thức truy xuất CSDL sẽ được định nghĩa thêm ở đây
8  }
9
10 module.exports = ContactService;
```

Hình 38: Khởi tạo lớp dịch vụ ContactService.

III. Cài đặt các handler

- Cài đặt handler create
- Hiệu chỉnh file **app/controllers/contact.controller.js**:

```

: > CT449 > Backend1 > app > controllers > JS contact.controller.js > ...
1  const ContactService = require("../services/contact.service");
2  const MongoDB = require("../utils/mongodb.util");
3  const ApiError = require("../api-error");
4
5  // Create and Save a new Contact
6  exports.create = async (req, res, next) => {
7      if (!req.body?.name) {
8          return next(new ApiError(400, "Name can not be empty"));
9      }
10
11     try {
12         const contactService = new ContactService(MongoDB.client);
13         const document = await contactService.create(req.body);
14         return res.send(document);
15     } catch (error) {
16         return next(
17             new ApiError(500, "An error occurred while creating the contact")
18         );
19     }
20 };

```

Hình 39: Cài đặt hàm xử lý tạo liên hệ trong controller.

– Cài đặt phương thức create trong Service

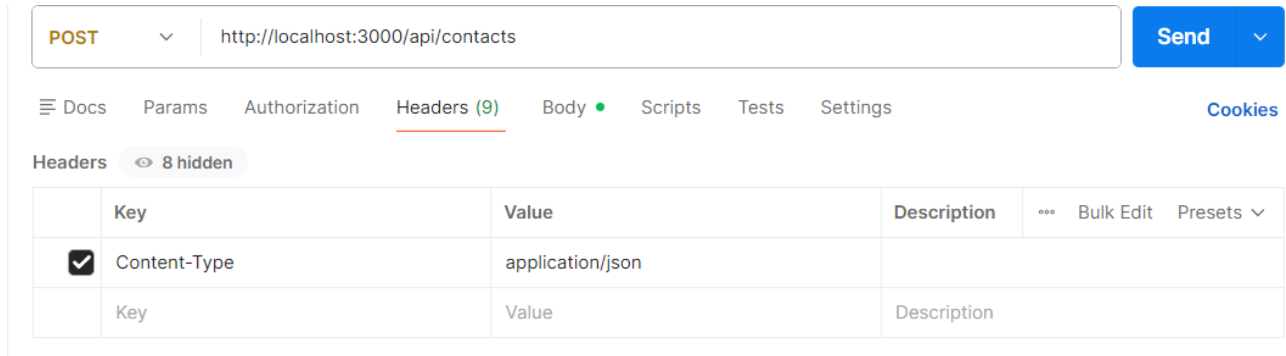
```

D: > CT449 > Backend1 > app > services > JS contact.service.js > ContactService > extractConactData
1  const { ObjectId } = require("mongodb");
2
3  class ContactService {
4      constructor(client) {
5          this.Contact = client.db().collection("contacts");
6      }
7
8      // Định nghĩa các phương thức truy xuất CSDL sử dụng mongodb API
9      extractConactData(payload) {
10         const contact = {
11             name: payload.name,
12             email: payload.email,
13             address: payload.address,
14             phone: payload.phone,
15             favorite: payload.favorite,
16         }; [cite: 105-111]
17
18         // Loại bỏ các trường có giá trị undefined
19         Object.keys(contact).forEach(
20             (key) => contact[key] === undefined && delete contact[key]
21         );
22         return contact;
23     }
24
25     async create(payload) {
26         const contact = this.extractConactData(payload);
27         const result = await this.Contact.findOneAndUpdate(
28             contact,
29             { $set: { favorite: contact.favorite === true } },
30             { returnDocument: "after", upsert: true }
31         );
32         return result;
33     }
34 }
35
36 module.exports = ContactService;

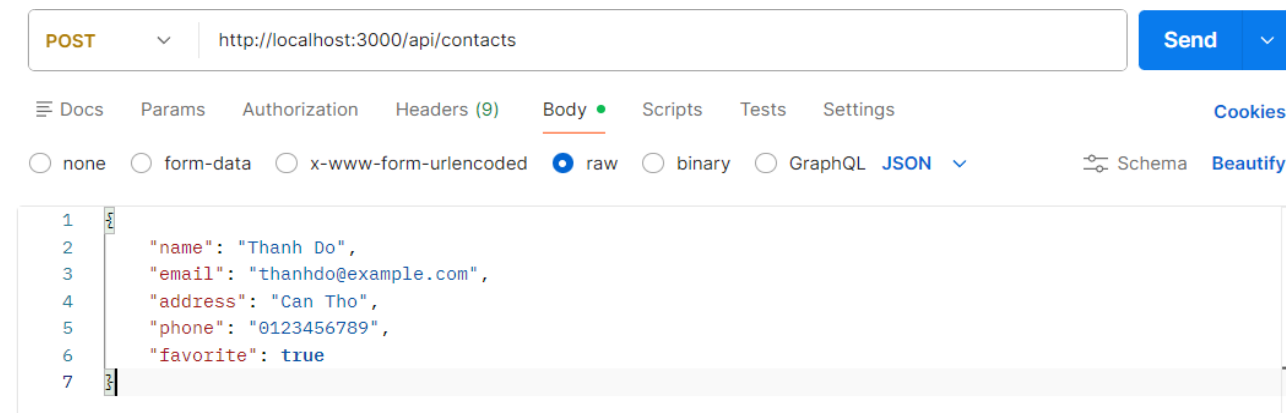
```

Hình 40: Xây dựng logic tạo liên hệ trong service.

- Dùng Postman hoặc curl kiểm tra handler hoạt động đúng



Hình 41: Thiết lập kiểu dữ liệu gửi đi trên Postman.



Hình 42: Kiểm tra tính năng tạo liên hệ bằng Postman.

- Cài đặt handler `findAll` trong tập tin `app/controllers/contact.controller.js`

```

exports.findAll = async (req, res, next) => {
  let documents = [];

  try {
    const contactService = new ContactService(MongoDB.client);
    const { name } = req.query;
    if (name) {
      documents = await contactService.findByName(name);
    } else {
      documents = await contactService.find({});
    }
  } catch (error) {
    return next(
      new ApiError(500, "An error occurred while retrieving contacts")
    );
  }

  return res.send(documents);
};

```

Hình 43: Cài đặt hàm lấy tất cả liên hệ trong controller.

- Định nghĩa hai phương thức *contactService.find(filter)* và *contactService.findByName(name)* trong tập tin *app/services/contact.service.js*

```
async find(filter) {
  const cursor = await this.Contact.find(filter);
  return await cursor.toArray();
}

async findByName(name) {
  return await this.find({
    name: { $regex: new RegExp(name), $options: "i" },
  });
}
```

Hình 44: Xây dựng logic tìm kiếm trong service.

- Kiểm tra bằng Postman

The screenshot shows the Postman interface. At the top, a GET request is configured to `http://localhost:3000/api/contacts?name=Thanh`. The 'Params' tab is active, showing a query parameter 'name' with the value 'Thanh'. Below this, the 'Body' tab is selected, displaying a JSON response. The response is a 200 OK status with a 5 ms response time and 409 B of data. The JSON body is an array containing one object with the following fields: `_id`, `name`, `email`, `address`, `favorite`, and `phone`.

Key	Value	Description
☒	name	Thanh

```
[
  {
    "_id": "69a1c06c9cb67cd4a0eb6c09",
    "name": "Thanh Do",
    "email": "thanhdo@example.com",
    "address": "Can Tho",
    "favorite": true,
    "phone": "0123456789"
  }
]
```

Hình 45: Kiểm tra tìm kiếm theo tên bằng Postman.

- Cài đặt handler `findOne` trong tập tin ***app/controllers/contact.controller.js***

```
exports.findOne = async (req, res, next) => {
  try {
    const contactService = new ContactService(MongoDB.client);
    const document = await contactService.findById(req.params.id);
    if (!document) {
      return next(new ApiError(404, "Contact not found"));
    }
    return res.send(document);
  } catch (error) {
    return next(
      new ApiError(500, `Error retrieving contact with id=${req.params.id}`)
    );
  }
};
```

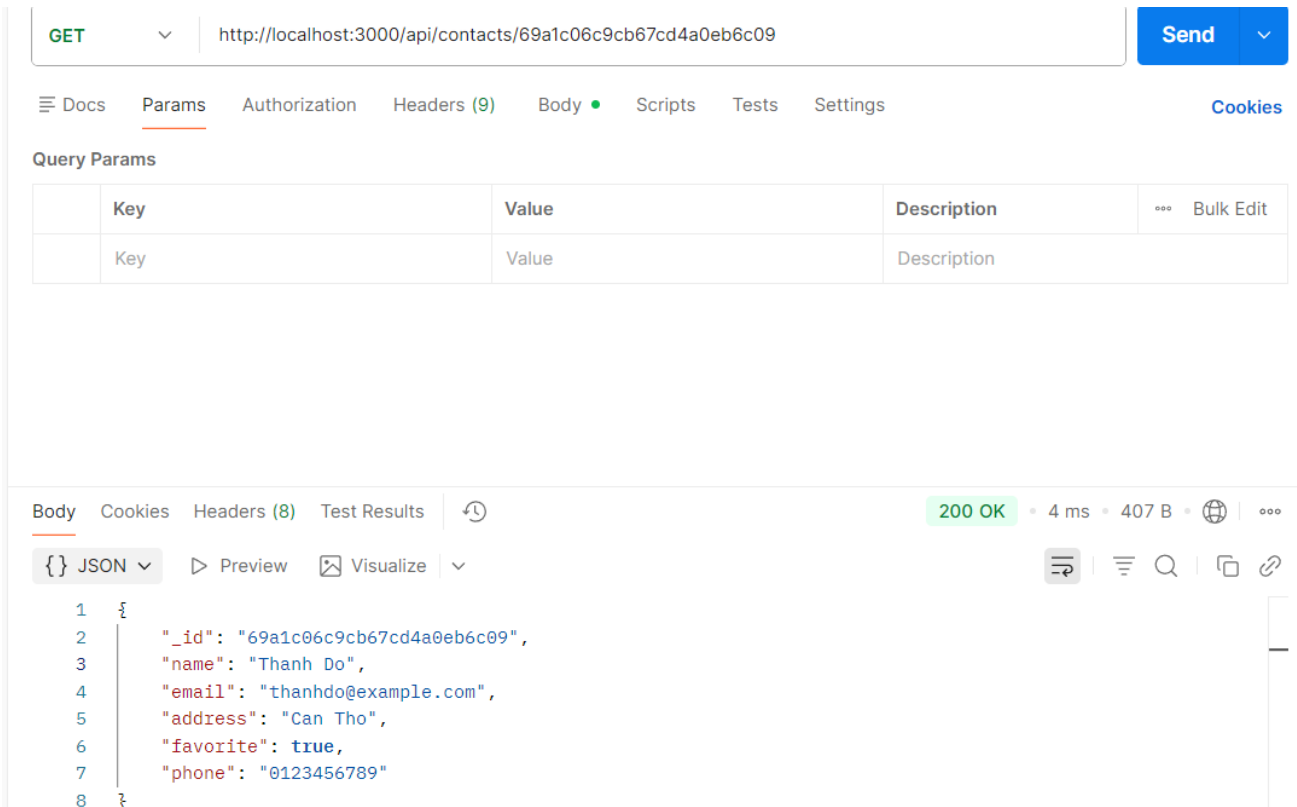
Hình 46: Cài đặt hàm lấy một liên hệ trong controller.

- Định nghĩa phương thức ***contactService.findById(id)*** trong tập tin ***app/services/contact.service.js***

```
async findById(id) {
  return await this.Contact.findOne({
    _id: ObjectId.isValid(id) ? new ObjectId(id) : null,
  });
}
```

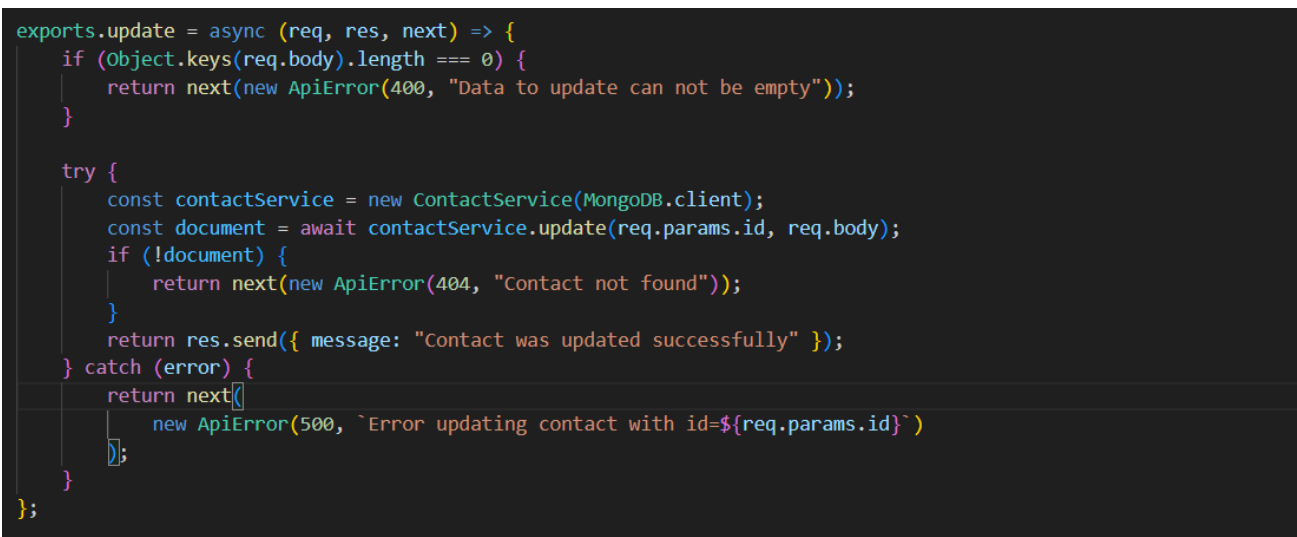
Hình 47: Xây dựng logic tìm kiếm theo ID trong service.

– Kiểm tra bằng Postman



Hình 48: Kiểm tra lấy chi tiết liên hệ bằng Postman.

– Cài đặt handler update trong tập tin `app/controllers/contact.controller.js`



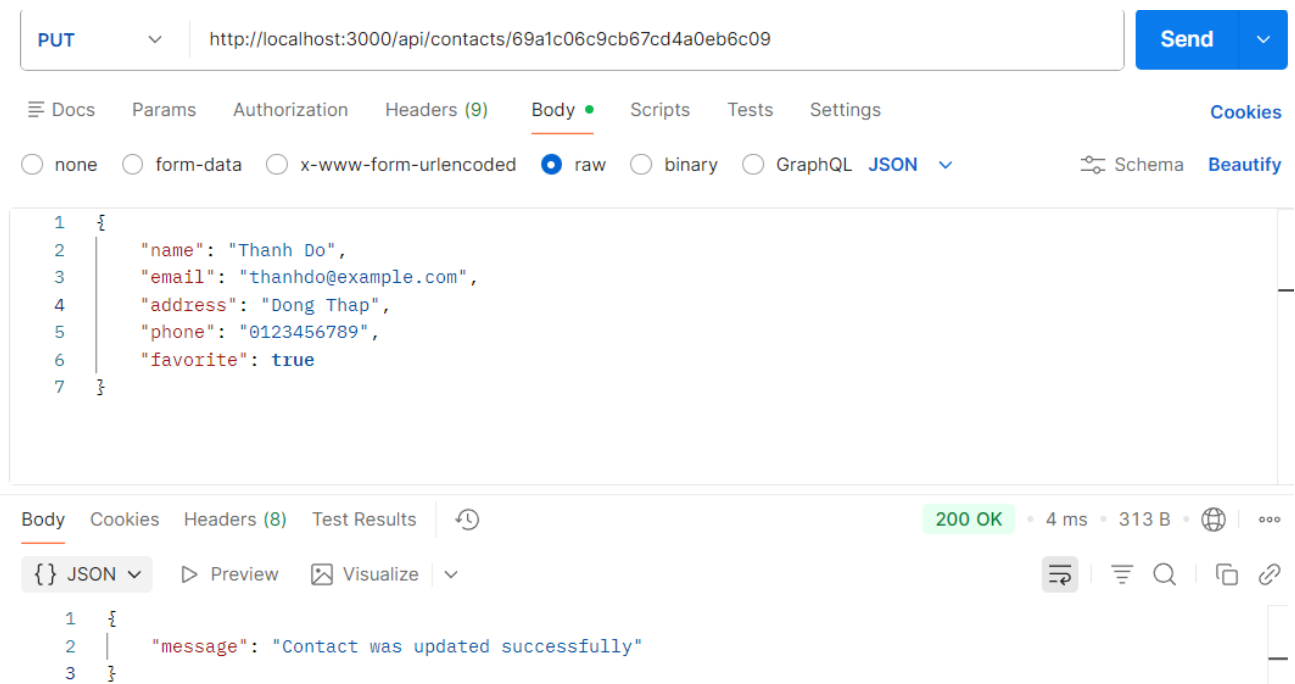
Hình 49: Cài đặt hàm cập nhật trong controller.

- Định nghĩa phương thức *contactService.update(id, payload)* trong tập tin *app/services/contact.service.js*

```
async update(id, payload) {
  const filter = {
    _id: ObjectId.isValid(id) ? new ObjectId(id) : null,
  };
  const update = this.extractContactData(payload);
  const result = await this.Contact.findOneAndUpdate(
    filter,
    { $set: update },
    { returnDocument: "after" }
  );
  return result;
}
```

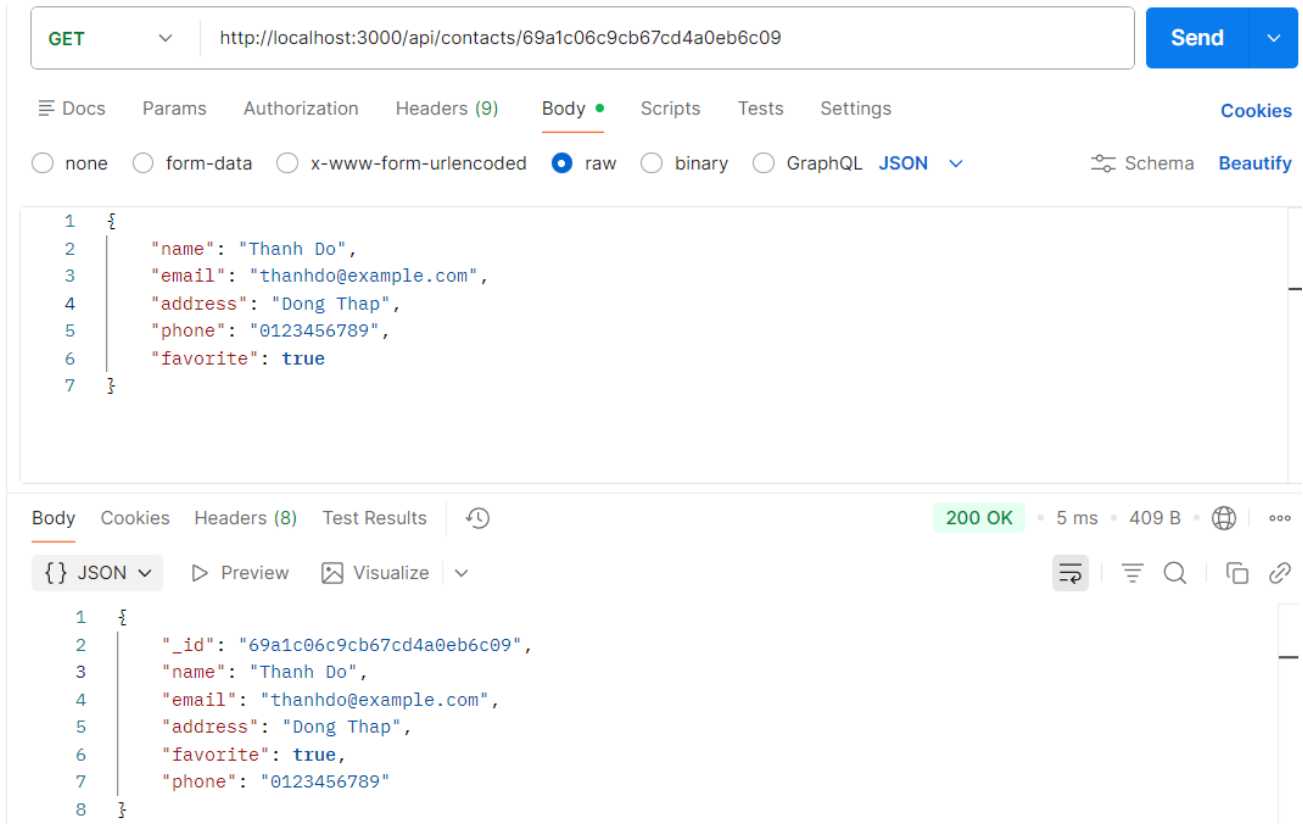
Hình 50: Xây dựng logic cập nhật dữ liệu trong service.

- Kiểm tra bằng Postman
 - Cập nhật thành công



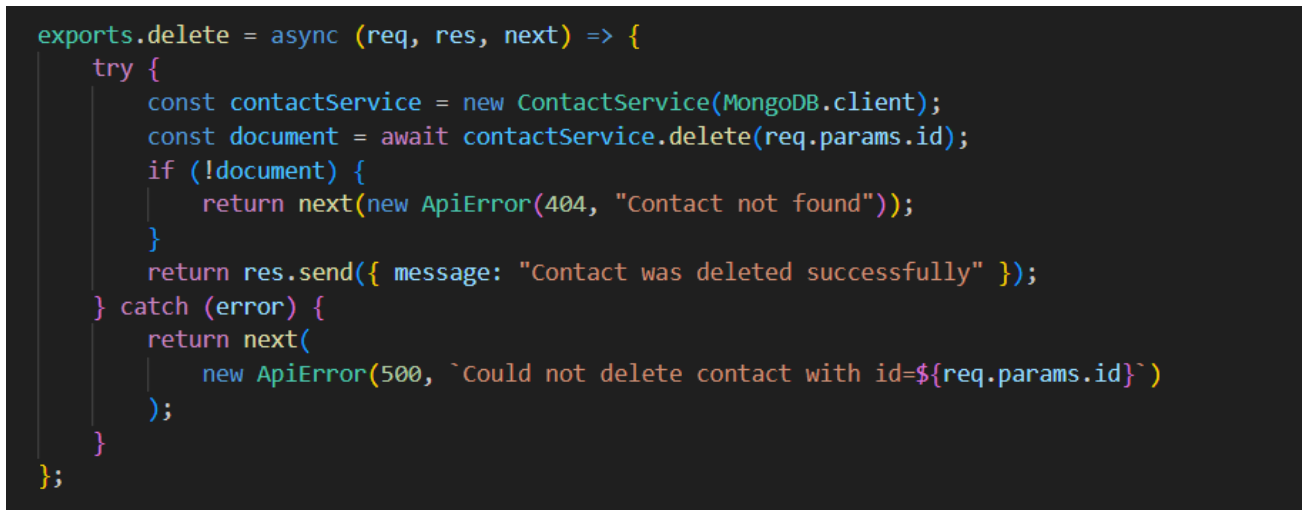
Hình 51: Thông báo cập nhật thành công trên Postman.

- Kết quả sau khi cập nhật



Hình 52: Kiểm tra dữ liệu sau khi cập nhật.

- Cài đặt handler delete trong tập tin *app/controllers/contact.controller.js*



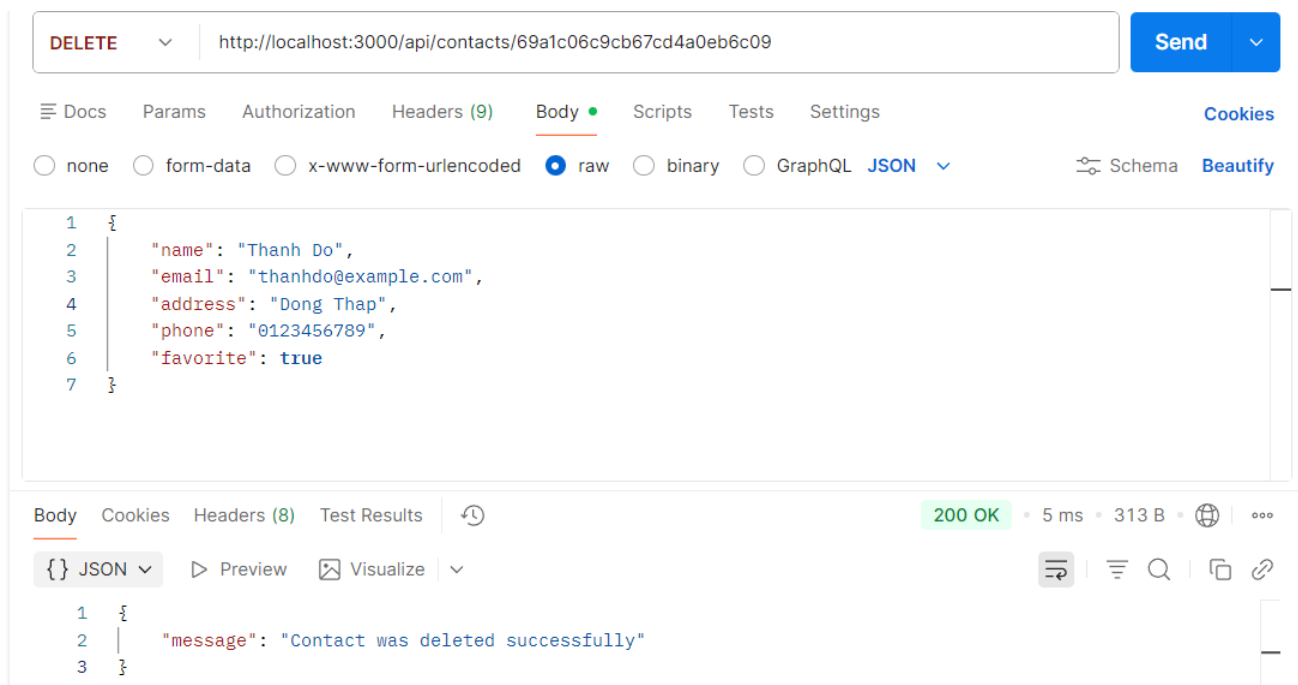
Hình 53: Cài đặt hàm xóa liên hệ trong controller.

- Định nghĩa phương thức *contactService.delete(id)* trong tập tin *app/services/contact.service.js*

```
async delete(id) {  
  const result = await this.Contact.findOneAndDelete({  
    _id: ObjectId.isValid(id) ? new ObjectId(id) : null,  
  });  
  return result;  
}
```

Hình 54: Xây dựng logic xóa dữ liệu trong service.

- Kiểm tra bằng Postman



Hình 55: Kiểm tra tính năng xóa bằng Postman.

- Cài đặt handler *findAllFavorite* trong tập tin *app/controllers/contact.controller.js*

```
exports.findAllFavorite = async (_req, res, next) => {  
  try {  
    const contactService = new ContactService(MongoDB.client);  
    const documents = await contactService.findFavorite();  
    return res.send(documents);  
  } catch (error) {  
    return next(  
      new ApiError(500, "An error occurred while retrieving favorite contacts")  
    );  
  }  
};
```

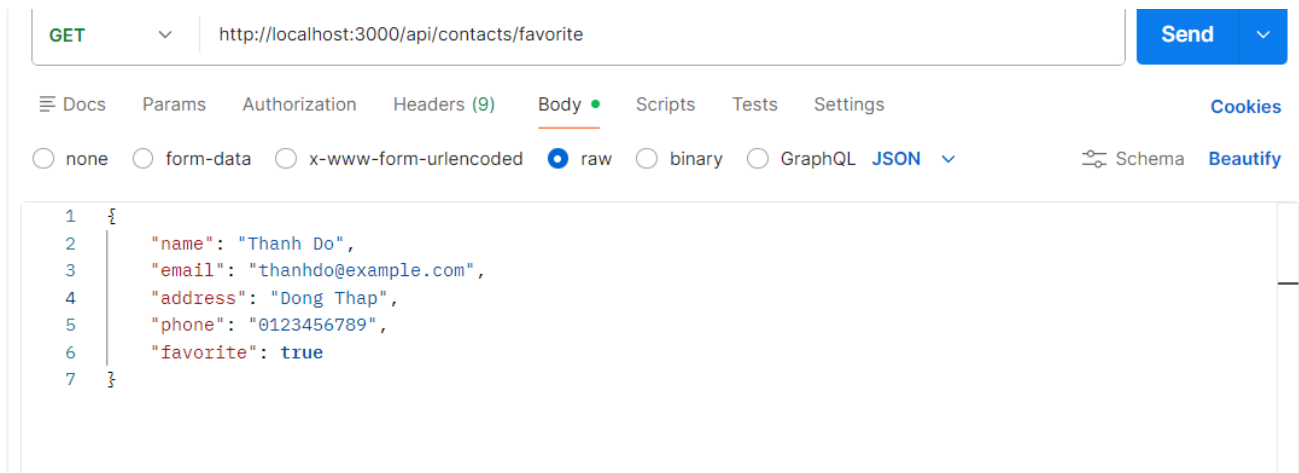
Hình 56: Cài đặt hàm lọc liên hệ yêu thích trong controller.

- Định nghĩa phương thức *contactService.findFavorite (id)* trong tập tin *app/services/contact.service.js*

```
async findFavorite() {  
  return await this.find({ favorite: true });  
}
```

Hình 57: Xây dựng logic tìm liên hệ yêu thích trong service.

- Kiểm tra bằng Postman



Hình 58: Kiểm tra danh sách yêu thích bằng Postman.

- Cài đặt handler deleteAll trong tập tin *app/controllers/contact.controller.js*

```
exports.deleteAll = async (_req, res, next) => {  
  try {  
    const contactService = new ContactService(MongoDB.client);  
    const deletedCount = await contactService.deleteAll();  
    return res.send({  
      message: `${deletedCount} contacts were deleted successfully`,  
    });  
  } catch (error) {  
    return next(  
      new ApiError(500, "An error occurred while removing all contacts")  
    );  
  }  
};
```

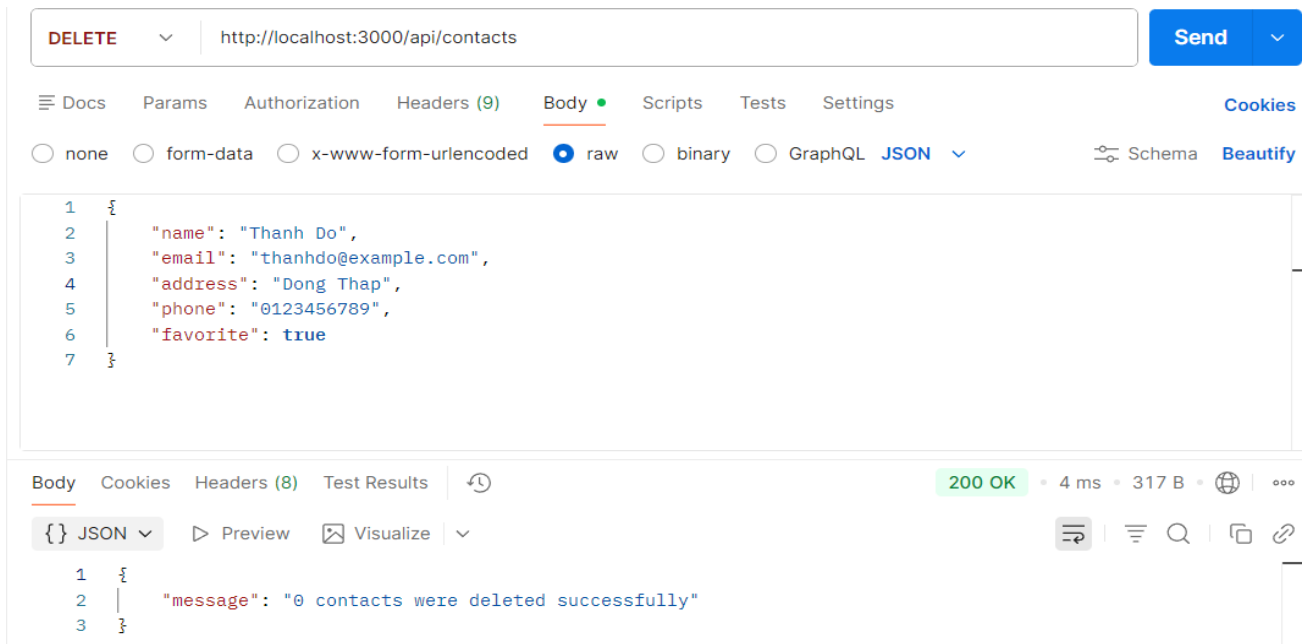
Hình 59: Cài đặt hàm xóa tất cả trong controller.

- Định nghĩa phương thức `contactService.deleteAll()` trong tập tin `app/services/contact.service.js`

```
async deleteAll() {  
    const result = await this.Contact.deleteMany({});  
    return result.deletedCount;  
}
```

Hình 60: Xây dựng logic xóa tất cả trong service.

- Kiểm tra bằng Postman



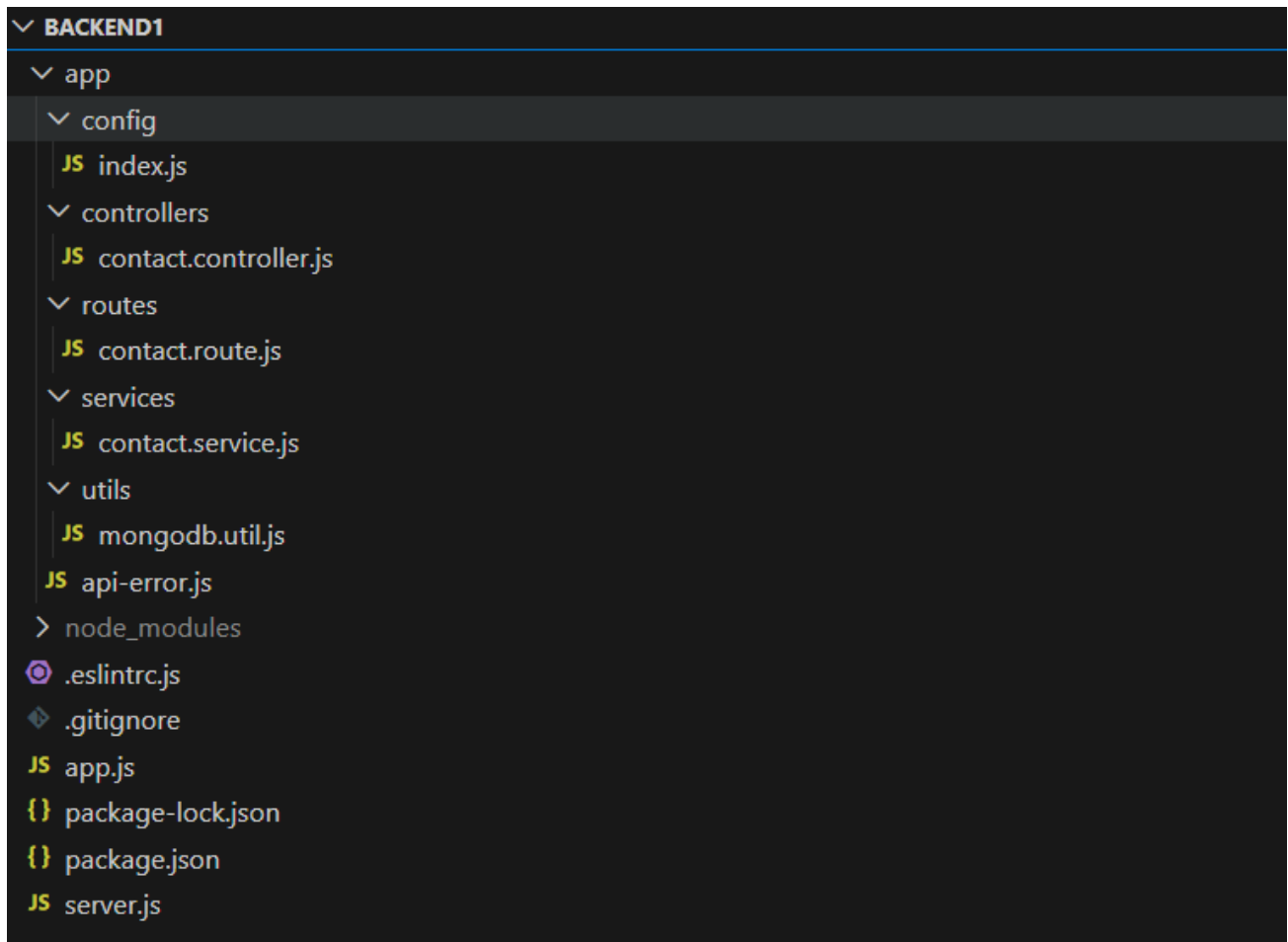
Hình 61: Kiểm tra tính năng xóa tất cả bằng Postman.

- Cập nhật những thay đổi lên GitHub

```
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)  
$ git add .  
  
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)  
$ git commit -m "Cai dat cac handler truy xuat CSDL"  
On branch master  
Your branch is up to date with 'origin/master'.  
  
nothing to commit, working tree clean  
  
Thanh Do@DESKTOP-Q0HDLAI MINGW64 /d/CT449/Backend1 (master)  
$ git push origin master  
Everything up-to-date
```

Hình 62: Cập nhật thay đổi phần Backend 2 lên GitHub.

- Cấu trúc thư mục dự án hiện tại:



Hình 63: Cấu trúc thư mục hoàn thiện của dự án.

LINK REPO BACKEND2:

https://github.com/thdo12362/B2306613_Lam_Thanh_Do_Backend2/tree/master